

PROJECT REPORT

STUDENT INFORMATION SYSTEM

Programming in Java

Submitted By:	Mayank Verma
Register Number:	25BAI10369
Program / Branch:	CSE (AI & ML)
Semester:	Fall Semester
Faculty:	Mo. Rizwan
Academic Year:	2026-27

1. Introduction

The Student Information System is a Java-based terminal application developed to manage student records in a simple, structured, and reliable manner. The application provides a menu-driven console interface through which the user can add, display, search, update, and delete student information.

The implementation uses Core Java concepts including classes and objects, encapsulation, ArrayList, methods, loops, conditional statements, exception handling, file handling, and object serialization. Student records are stored locally in students.dat so that saved information can be restored when the application is started again.

2. Problem Statement

Manual management of student records can become difficult as the number of records increases. It may result in duplicate IDs, incorrect GPA values, difficulty locating a student, and loss of information. The proposed system provides a terminal-based solution for maintaining student ID, name, department, and GPA with validation and persistent local storage.

3. Functional Requirements

ID	Requirement
FR-01	Add a new student record.
FR-02	Prevent duplicate student IDs.
FR-03	Display all registered students.
FR-04	Search a student using ID.
FR-05	Update name, department, and optionally GPA.
FR-06	Delete a student after confirmation.
FR-07	Accept GPA only from 0.0 to 4.0.
FR-08	Handle invalid integer and decimal input.
FR-09	Load saved records from students.dat.
FR-10	Save records using Java serialization when exiting.

4. Non-functional Requirements

- **Usability:** Clear menu-driven terminal interaction.
- **Reliability:** Invalid input is handled using validation and exception handling.
- **Maintainability:** Operations are separated into methods.
- **Performance:** Suitable response time for a small student dataset.
- **Portability:** Runs on systems with a compatible Java JDK.
- **Persistence:** Records remain available after saving and restarting the application.

5. System Architecture

The application follows a simple console-based architecture consisting of user interaction, application logic, in-memory data, and file storage.

Architecture flow: User Console Menu → StudentManagementSystem → ArrayList<Student> → students.dat

6. Design Diagrams & Models

6.1 Use Case & Workflow Summary

The user is the primary actor and can perform all major system operations (Add, Display, Search, Update, Delete, Save & Exit) through the terminal menu loop.

6.2 Data Model (ER / Entity Structure)

The project does not use a relational database. It stores serialized Student objects. The logical data model consists of the Student entity with attributes: id (unique student identifier), name (full name), department (academic department), and gpa (value from 0.0 to 4.0). Persistence is handled via students.dat.

7. Design Decisions & Rationale

Decision	Rationale
Terminal interface	Keeps the project focused on Java fundamentals without GUI complexity.
ArrayList<Student>	Provides simple dynamic storage for a small dataset.
Serializable	Allows Student objects and the list to be stored and restored.
students.dat	Provides local persistence without an external database.
Case-insensitive ID search	Makes student ID lookup more convenient.
GPA validation	Ensures GPA remains within the accepted 0.0-4.0 range.
Separate methods	Improves readability, modularity, and maintainability.

8. Implementation Details

Student class: Implements Serializable and stores four private fields: id, name, department, and gpa. A constructor initializes each object. Getter and setter methods provide controlled access, while toString() formats a record for terminal display.

StudentManagementSystem class: Maintains an ArrayList of Student objects and a Scanner. The main() method loads saved data, displays the menu, reads the user's choice, and invokes the selected operation.

CRUD operations: addStudent() creates a record after checking duplicate IDs; viewAllStudents() displays records; searchStudent() finds a record by ID; updateStudent() changes selected fields; deleteStudent() removes a record after confirmation.

Input validation & Persistence: readIntInput() and readDoubleInput() ensure correct values. loadData() and saveData() handle ObjectInputStream and ObjectOutputStream serialization.

9. Testing Approach

Testing focuses on normal operations, invalid inputs, boundary values, and data persistence.

TC	Test Input / Action	Expected Result
TC-01	Valid new student	Record added successfully.
TC-02	Existing student ID	Duplicate ID rejected.
TC-03	GPA below 0 or above 4	Value rejected and input requested again.
TC-04	Display all students	All records displayed.
TC-05	Existing ID search	Matching record displayed.
TC-06	Unknown ID search	No record found message.
TC-07	Update existing record	Selected fields updated.
TC-08	Delete + y confirmation	Record deleted.
TC-09	Delete + n confirmation	Deletion cancelled.
TC-10	Restart after Save & Exit	Saved records loaded from students.dat.
TC-11	Non-numeric menu input	Input error shown; program continues.

10. Challenges Faced & Learnings

Challenges: Handling invalid numeric input without crashing, preventing duplicate student IDs, updating/deleting objects inside an ArrayList, and implementing Java object serialization.

Learnings: Successfully applied object-oriented programming in Java, utilized encapsulation, managed dynamic collections with ArrayList, implemented file I/O streams, and organized a complete project structure.

11. Future Enhancements & Conclusion

Future Enhancements: Add role-based authentication, integrate MySQL database persistence, add filtering/sorting features, export to CSV/PDF, and develop a graphical or web-based frontend interface.

Conclusion: The Student Information System demonstrates how Core Java can be used to develop a practical terminal-based record-management application supporting CRUD operations, input validation, and persistent storage.